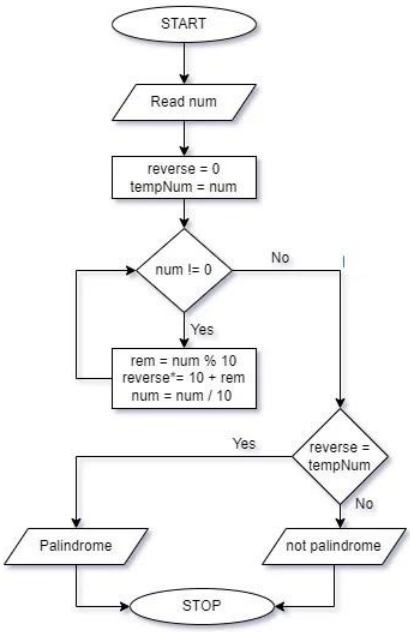
I Semester BTech Examination Nov-Dec 2025	Set No.: 02
Course name: Programming for Problem Solving	Course code: CSS 1001

Q. No.	Description	Marks	Course Outcome (1-5)	Competency Levels (1-6)
1A	Design an algorithm and create a flowchart to check if a given number is a palindrome. Sol: Step 1: Start Step 2: Read the input number from the user Step 3: Declare and initialize the variable reverse and assign input to a temp variable tempNum=num Step 4: Start the while loop until num !=0 becomes false 4.1 rem = num % 10 4.2 reverse*= 10 + rem 4.3 num = num / 10 Step 5 : Check if reverse == tempNum Step 6: If it's true then the number is a palindrome Step 7: If not, the number is NOT a palindrome Step 8: Stop  Evaluation Scheme: Flowchart - 1 Mark Algorithm -1 Mark	2	1	3

1B	Identify and explain the different types of computer languages, highlighting their unique characteristics and examples for each.
	
	Sol:
	• Machine Language- The only programming language available in earlier days
	  • Consists of only 0's and 1's; e.g.:- 10101011
	• Symbolic language or Assembly language-
	  • symbols or mnemonics are used to represent instructions
	  • hardware specific
		3	1	2

	• e.g. MASM : ADD X,Y; Add the contents of y to x • High-level languages- English like language using which the programmer can write programs to solve a problem. • more concerned with the problem specification • not oriented towards the details of computer • e.g.: C, C++, C#, Fortran, BASIC, Pascal etc. Evaluation Scheme: 3 different types of languages – 1 mark each			

1C	i) Write a C program that generates a grade based on the score in a subject using a switch statement. The program should take an integer input (0 to 100) representing the student's score and should generate and display grades based on following criteria.
	• 90 and above: Grade A
	• 80 to 89: Grade B
	• 70 to 79: Grade C
	• 60 to 69: Grade D
	• Below 60: Grade F
	
	ii) Referring to the criteria in part (i) question above, write a C program to generate and display grades, using conditional (ternary) operators only [do not use if's and switch].
	
	iii) Evaluate the following expression and provide the resultant: Consider: i = 9, j = 5.
	(i * 3 + j % i + j - 1 + i - 4 / j + 1)
	
	Sol:
	(i). #include <stdio.h>
	
	int main() {
	int score;
	char grade;
	
	// Input score
	printf("Enter the student's score (0 to 100): ");
	scanf("%d", &score);
	
	// Check if the score is within the valid range
	if (score < 0 || score > 100) {
	printf("Invalid score. Please enter a number between 0 and 100.\n");
	return 1; // Exit the program with an error code
	}
	
	// Determine the grade using switch
	switch (score / 10) { // Divide by 10 to get the range
	case 10: // for scores 100
		2+2+1	2	3

```
            case 9:
               grade = 'A';
               break;
            case 8:
               grade = 'B';
               break;
            case 7:
               grade = 'C';
               break;
            case 6:
               grade = 'D';
               break;
            default:
               grade = 'F';
               break;
         }

         // Display the grade
         printf("The grade for a score of %d is: %c\n",
      score, grade);

         return 0;
}
```

ii)
```
#include <stdio.h>

int main() {
   int marks;
   char grade;

   // Input marks from user
   printf("Enter the student's marks: ");
   scanf("%d", &marks);

   // Using nested ternary operator to assign grade
   grade = (marks >= 90) ? 'A' :
         (marks >= 80) ? 'B' :
         (marks >= 70) ? 'C' :
         (marks >= 60) ? 'D' : 'F';

   // Display the result
   printf("Grade: %c\n", grade);

   return 0;
         }
```

iii) Consider: i = 9, j = 5.
(i * 3 + j % i + j - 1 + i - 4 / j + 1)

9 * 3 + 5 % 9 + 5 - 1 + 9 - 4 / 5 + 1
(9 * 3) + (5 % 9) + 5 - 1 + 9 – (4 / 5) + 1
27 + 5 + 5 – 1 + 9 – 0 + 1 = 46

Scheme of evaluation

(i) **I/O** - 1 Mark, **Use of Switch Statement** - 1 Mark
(ii) **I/O** - 1 Mark, **Use of nested ternary op.** - 1 Mark
(iii) Correct answer is **46** – 1 mark

2A	Write a C program utilizing appropriate decision-making statements to compute the commission based on the input sales amount, considering the following conditions:
	
	a. For sales amounts less than or equal to Rs. 5000 no commission is given to females, males receive a commission of 2%.
	
	b. For sales amounts greater than Rs. 5000 and up to Rs. 10,000 females receive a commission of 2%, males receive a commission of 4%.
	
	c. For sales amounts exceeding Rs. 10,000, both males and females receive a commission of 6%.
	
	Sol:
	#include <stdio.h>
	
	int main() {
	float sales_amount, commission = 0;
	char gender;
	
	// Input sales amount and gender
	printf("Enter sales amount: Rs. ");
	scanf("%f", &sales_amount);
	printf("Enter gender (M/F): ");
	scanf(" %c", &gender); // Note the space before %c to skip whitespace
	
	printf("\n=== Using Nested If Structure ===\n");
	
	// Calculate commission using nested if
	if (sales_amount > 10000) {
	commission = sales_amount * 0.06;
	} else {
	if (gender == 'F' || gender == 'f') {
	if (sales_amount <= 5000) {
	commission = 0;
	} else {
	commission = sales_amount * 0.02;
	}
	} else if (gender == 'M' || gender == 'm') {
	if (sales_amount <= 5000) {
	commission = sales_amount * 0.02;
	} else {
	commission = sales_amount * 0.04;
	}
	} else {
	printf("Invalid gender entered!\n");
	return 1;
	}
	}
	
	printf("Commission (Nested If): Rs. %.2f\n", commission);
		2	2	3

	// Reset commission for else-if ladder calculation
	commission = 0;
	
	printf("\n=== Using Else-If Ladder Structure ===\n");
	
	// Calculate commission using else-if ladder
	if (sales_amount > 10000) {
	commission = sales_amount * 0.06;
	}
	else if ((gender == 'F' || gender == 'f') && sales_amount <= 5000) {
	commission = 0;
	}
	else if ((gender == 'F' || gender == 'f') && sales_amount > 5000) {
	commission = sales_amount * 0.02;
	}
	else if ((gender == 'M' || gender == 'm') && sales_amount <= 5000) {
	commission = sales_amount * 0.02;
	}
	else if ((gender == 'M' || gender == 'm') && sales_amount > 5000) {
	commission = sales_amount * 0.04;
	}
	else {
	printf("Invalid gender entered!\n");
	return 1;
	}
	
	printf("Commission (Else-If Ladder): Rs. %.2f\n", commission);
	
	return 0;
	}
	
	Scheme: input/output -1 M logic 1 M			

2B	Write a complete C program to read a number and check for the following operations using a while Loop.
	• Display "EVEN" if all the digits in the number are even.
	• Display "ODD" if all the digits in the number are odd.
	• Display "neither EVEN nor ODD" if all the digits in the number are neither even nor odd.
	
	Sol:
	#include <stdio.h>
	
	int main() {
	int number, temp;
	int hasEven = 0, hasOdd = 0;
		3	2	3

```
    // Input the number
    printf("Enter a number: ");
    scanf("%d", &number);

    // Store original number for printing later
    temp = number;

    // Handle negative numbers
    if (number < 0) {
        number = -number;
    }

    // Handle zero separately
    if (number == 0) {
        printf("%d contains all EVEN digits\n", temp);
        return 0;
    }

    // Check each digit using while loop
    while (number > 0) {
        int digit = number % 10;  // Get last digit

        // Check if digit is even or odd
        if (digit % 2 == 0) {
            hasEven = 1;
        } else {
            hasOdd = 1;
        }

        // Break early if we found both even and odd
digits
        if (hasEven && hasOdd) {
            break;
        }

        number = number / 10;  // Remove last digit
    }

    // Print result based on what types of digits were
found
    if (hasEven && !hasOdd) {
        printf("%d contains all EVEN digits\n", temp);
    }
    else if (!hasEven && hasOdd) {
        printf("%d contains all ODD digits\n", temp);
    }
    else {
        printf("%d is neither EVEN nor ODD (contains
both even and odd digits)\n", temp);
    }

    return 0;
}
```

Scheme: input/output-1 M, Logic 2 M

2C	Explain the significance of a 1D array in programming and its practical applications. Discuss various methods for initializing an array. Additionally, write a C program that reads N elements into a single array and then separates the elements into two arrays: one containing prime numbers and the other containing non-prime numbers.
	
	Sol:
	Significance of a 1D array in programming and its practical applications
	
	In the C programming language an array is a fixed sequenced collection of elements of the same data type. It is simply a grouping of like type data. Array provides a convenient structure for representing data. (Any other related explanation can be awarded) **1M**
	
	Initializing an array
	
	int number[3] ={ 0,0,0}; or {0} ;
	int age[ ] ={16,25,32,48,52,65};
	values[i]=0; **1M**
	
	Program
	#include <stdio.h>
	
	int main() {
	int N;
	
	// Input array size
	printf("Enter the number of elements: ");
	scanf("%d", &N);
	
	// Declare arrays
	int arr[100]; // Original array
	int primes[100]; // Array for prime numbers
	int non_primes[100]; // Array for non-prime numbers
	int prime_count = 0, non_prime_count = 0;
	
	// Input array elements
	printf("Enter %d elements:\n", N);
	for (int i = 0; i < N; i++) {
	scanf("%d", &arr[i]);
	}
	
	// Split array into prime and non-prime
	for (int i = 0; i < N; i++) {
	int num = arr[i];
	int isPrime = 1; // 1 means prime, 0 means not prime
	// Check if number is prime
	if (num <= 1) {
	isPrime = 0;
	}
	else if (num == 2) {
	isPrime = 1;
	}
		5	3	4

```
        else if (num % 2 == 0) {
            isPrime = 0;
        }
        else {
            for (int j = 3; j * j <= num; j += 2) {
                if (num % j == 0) {
                    isPrime = 0;
                    break;
                }
            }
        }

        // Add to appropriate array
        if (isPrime) {
            primes[prime_count] = num;
            prime_count++;
        } else {
            non_primes[non_prime_count] = num;
            non_prime_count++;
        }
    }

    // Print prime numbers array
    printf("\nPrime numbers: ");
    if (prime_count == 0) {
        printf("None");
    } else {
        for (int i = 0; i < prime_count; i++) {
            printf("%d ", primes[i]);
        }
    }

    // Print non-prime numbers array
    printf("\nNon-prime numbers: ");
    if (non_prime_count == 0) {
        printf("None");
    } else {
        for (int i = 0; i < non_prime_count; i++) {
            printf("%d ", non_primes[i]);
        }
    }

    printf("\n");
    return 0;
}
```

Scheme: Input and output 1M, logic 2M

3A	Write a C program to count the number of alphabets, digits, and special characters in a given string.
	
	Sol:
	#include <stdio.h>
	
	int main() {
	char str[100];
	int i, alphabets = 0, digits = 0, special = 0;
	printf("Enter a string: ");
	gets(str); // reads the input string **-0.5-**
	for(i = 0; str[i] != '\0'; i++) {
		2	3	3

	if((str[i] >= 'A' && str[i] <= 'Z') ‖ (str[i] >= 'a' && str[i] <= 'z'))
	alphabets++;
	else if(str[i] >= '0' && str[i] <= '9')
	digits++;
	else
	special++;
	} **-1-**
	
	printf("\nAlphabets = %d", alphabets);
	printf("\nDigits = %d", digits);
	printf("\nSpecial Characters = %d", special);
	
	return 0;
	} **-0.5-**			

3B	Write a C program to read the elements of a 3 × 3 matrix and sort the elements of each row in ascending order. Display the matrix after sorting each row.
	
	Sol:
	#include <stdio.h>
	
	int main() {
	int a[3][3], i, j, k, temp;
	
	printf("Enter elements of 3x3 matrix:\n");
	for(i=0; i<3; i++)
	for(j=0; j<3; j++)
	scanf("%d", &a[i][j]); **-0.5-**
	
	// Sorting each row in ascending order
	for(i=0; i<3; i++) {
	for(j=0; j<3-1; j++) {
	for(k=j+1; k<3; k++) {
	if(a[i][j] > a[i][k]) {
	temp = a[i][j];
	a[i][j] = a[i][k];
	a[i][k] = temp;
	}
	}
	}
	} **-1.5-**
	
	printf("\nMatrix after sorting each row in ascending order:\n");
	for(i=0; i<3; i++) {
	for(j=0; j<3; j++)
	printf("%d ", a[i][j]);
	printf("\n");
	}
	
	return 0;
	} **-1-**
		3	3	3

3C	Design and implement a menu-driven C program using functions (without using structures) to manage student records. The program should allow the user to: 1. Add new student details (Name, Roll Number, Marks) 2. Display all student details 3. Search for a student by roll number 4. Calculate and display the class average marks Sol:	5	4	6

```
#include <stdio.h>
#include <string.h>

int main() {
   int rollNo[100];
   char name[100][50];
   float marks[100];
   int n = 0;  // Number of students
   int choice;                                    -0.5-

   void addStudent() {
      printf("Enter Roll Number: ");
      scanf("%d", &rollNo[n]);
      printf("Enter Name: ");
      scanf(" %[^\n]", name[n]);
      printf("Enter Marks: ");
      scanf("%f", &marks[n]);
      n++;
      printf("Student added successfully.\n");
   }                                              -1-

   void displayAll() {
      if (n == 0) {
         printf("No student records available.\n");
         return;
      }
      printf("\nStudent Records:\n");
      for (int i = 0; i < n; i++) {
         printf("Roll Number: %d\n", rollNo[i]);
         printf("Name: %s\n", name[i]);
         printf("Marks: %.2f\n", marks[i]);
         printf("--------------------\n");
      }
   }                                              -1-

   void searchStudent() {
      if (n == 0) {
         printf("No student records to search.\n");
         return;
      }
      int roll, found = 0;
      printf("Enter Roll Number to search: ");
      scanf("%d", &roll);
```

```
        for (int i = 0; i < n; i++) {
            if (rollNo[i] == roll) {
                printf("\nRecord Found:\n");
                printf("Roll Number: %d\n", rollNo[i]);
                printf("Name: %s\n", name[i]);
                printf("Marks: %.2f\n", marks[i]);
                found = 1;
                break;
            }
        }
        if (!found)
            printf("Record not found.\n");
    }
```
-1-

```
    void classAverage() {
        if (n == 0) {
            printf("No records to calculate
average.\n");
            return;
        }
        float sum = 0;
        for (int i = 0; i < n; i++)
            sum += marks[i];
        printf("Class Average Marks: %.2f\n", sum /
n);
    }
```
-0.5-

```
    do {
        printf("\n== Student Record Menu ==\n");
        printf("1. Add Student\n");
        printf("2. Display All Students\n");
        printf("3. Search Student by Roll
Number\n");
        printf("4. Calculate Class Average\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addStudent(); break;
            case 1: displayAll(); break;
            case 3: searchStudent(); break;
            case 4: classAverage(); break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice. Please try
again.\n");
        }
    } while (choice != 5);

    return 0;
}
```
-1-

4A	Write a C program using functions to calculate Simple Interest (SI), Compound Interest (CI), and their Difference. The program should take the principal, rate, and time as inputs and use separate functions for each calculation - one for SI, one for CI, and one for finding the difference.	3	4	3

	Demonstrate how using multiple functions improves the program's modularity, readability, and reusability.
	Hint: SI = (P × R × T) / 100,
	CI = P ((1 + R / 100)^T) – P
	
	Sol:
	#include <stdio.h>
	#include <math.h>
	
	// Function to calculate Simple Interest
	float calculateSI(float P, float R, float T) {
	  return (P * R * T) / 100.0;
	}
	
	// Function to calculate Compound Interest
	float calculateCI(float P, float R, float T) {
	  float amount = P * pow((1 + R / 100.0), T);
	  return amount - P;
	}
	
	// Function to calculate difference between CI and SI
	float difference(float CI, float SI) {
	  return CI - SI;
	}
	
	int main() {
	  float P, R, T, si, ci, diff;
	
	  // Input from the user
	  printf("Enter Principal: ");
	  scanf("%f", &P);
	
	  printf("Enter Rate of Interest: ");
	  scanf("%f", &R);
	
	  printf("Enter Time (in years): ");
	  scanf("%f", &T);
	
	  // Function calls
	  si = calculateSI(P, R, T);
	  ci = calculateCI(P, R, T);
	  diff = difference(ci, si);
	
	  // Output
	  printf("\nSimple Interest: %.2f", si);
	  printf("\nCompound Interest: %.2f", ci);
	  printf("\nDifference (CI - SI): %.2f\n", diff);
	
	  return 0;
	}
	Scheme: I/O –1M, 3 function calls and definitions – 2M			

4B	Design a program using functions with pointer parameters to compute both row-wise and column-wise sums of a matrix.	3	4	3

Sol:

```
#include <stdio.h>

void rowWiseSum(int *mat, int rows, int cols, int
*rowSum) {
    for (int i = 0; i < rows; i++) {
        rowSum[i] = 0;
        for (int j = 0; j < cols; j++) {
            rowSum[i] += *(mat + i * cols + j);  // pointer
arithmetic
        }
    }
}

void colWiseSum(int *mat, int rows, int cols, int
*colSum) {
    for (int j = 0; j < cols; j++) {
        colSum[j] = 0;
        for (int i = 0; i < rows; i++) {
            colSum[j] += *(mat + i * cols + j);  // pointer
arithmetic
        }
    }
}

int main() {
    int rows, cols;

    // Input for matrix size
    printf("Enter number of rows: ");
    scanf("%d", &rows);

    printf("Enter number of columns: ");
    scanf("%d", &cols);

    int mat[rows][cols];
    int rowSum[rows], colSum[cols];

    // Read matrix elements
    printf("\nEnter the matrix elements:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    // Function calls using pointers
    rowWiseSum((int *)mat, rows, cols, rowSum);
    colWiseSum((int *)mat, rows, cols, colSum);

    // Display row-wise sum
    printf("\nRow-wise Sums:\n");
    for (int i = 0; i < rows; i++) {
        printf("Row %d Sum = %d\n", i + 1,
rowSum[i]);
```

	``
	}
	
	// Display column-wise sum
	printf("\nColumn-wise Sums:\n");
	for (int j = 0; j < cols; j++) {
	printf("Column %d Sum = %d\n", j + 1,
	colSum[j]);
	}
	
	return 0;
	}
	``
	Scheme: I/O: 1M, Functions – 1M each			

4C	Write a C program using separate functions to perform operations on an array. Implement readArray() to input n integers, reverseArray() to reverse the array elements, findMaximum() to find the maximum value in the reversed array, and displayResult() to print both the reversed array and the maximum element. Each function should handle its task independently, and data should be passed appropriately between functions.
	
	Sol:
	``
	#include <stdio.h>
	
	// Function to read array elements
	void readArray(int arr[], int n) {
	printf("Enter %d integers:\n", n);
	for (int i = 0; i < n; i++) {
	scanf("%d", &arr[i]);
	}
	}
	
	// Function to reverse the array
	void reverseArray(int arr[], int n) {
	int start = 0, end = n - 1, temp;
	while (start < end) {
	temp = arr[start];
	arr[start] = arr[end];
	arr[end] = temp;
	
	start++;
	end--;
	}
	}
	
	// Function to find maximum element in the array
	int findMaximum(int arr[], int n) {
	int max = arr[0];
	for (int i = 1; i < n; i++) {
	if (arr[i] > max)
	max = arr[i];
	}
	return max;
	}
	``
	
		4	4	3

	// Function to display reversed array and maximum value
	void displayResult(int arr[], int n, int max) {
	  printf("\nReversed Array:\n");
	  for (int i = 0; i < n; i++) {
	  printf("%d ", arr[i]);
	  }
	
	  printf("\n\nMaximum Element: %d\n", max);
	}
	
	int main() {
	  int n;
	
	  // Input array size
	  printf("Enter number of elements: ");
	  scanf("%d", &n);
	
	  int arr[n];
	
	  // Function calls
	  readArray(arr, n);
	  reverseArray(arr, n);
	  int max = findMaximum(arr, n);
	  displayResult(arr, n, max);
	
	  return 0;
	}
	Scheme: 1M for each function			

5A	A shop maintains information about its products, including product_name, quantity, and price.
	You are required to use an array of structures and pointer arithmetic in C to perform the following task and display the details (product_name, quantity, and price) of products whose quantity is less than 5.:
	i) Calculate the total stock value of all products (where stock value of each product = quantity × price).
	
	Sol:
	#include <stdio.h>
	
	// Structure definition
	struct Product {
	  char name[50];
	  int quantity;
	  float price;
	};
	
	// Function to read product details
	void readProducts(struct Product *p, int n) {
	  for (int i = 0; i < n; i++) {
	  printf("\nEnter details of product %d\n", i + 1);
	  printf("Product Name: ");
	  scanf("%s", (p + i)->name);
		3	5	3

```
        printf("Quantity: ");
        scanf("%d", &(p + i)->quantity);
        printf("Price: ");
        scanf("%f", &(p + i)->price);
    }
}

// Function to calculate total stock value using
pointer arithmetic
float totalStockValue(struct Product *p, int n) {
    float total = 0;
    for (int i = 0; i < n; i++) {
        total += (p + i)->quantity * (p + i)->price;
    }
    return total;
}

// Function to display products with quantity < 5
void displayLowStock(struct Product *p, int n) {
    printf("\nProducts with quantity less than 5:\n");
    printf("-------------------------------------\n");
    for (int i = 0; i < n; i++) {
        if ((p + i)->quantity < 5) {
            printf("Name: %s\n", (p + i)->name);
            printf("Quantity: %d\n", (p + i)->quantity);
            printf("Price: %.2f\n\n", (p + i)->price);
        }
    }
}

int main() {
    int n;

    printf("Enter number of products: ");
    scanf("%d", &n);

    struct Product products[n];

    // Read product details
    readProducts(products, n);

    // Compute total stock value
    float stockValue = totalStockValue(products, n);

    // Display results
    printf("\nTotal Stock Value of All Products:
%.2f\n", stockValue);

    displayLowStock(products, n);

    return 0;
}
```

Scheme: Structure definition – 0.5M, I/O – 1M, Logic – 2M

5B	Analyze the difference in accessing structure members using a normal structure variable and a pointer to structure. Write a C program showing both methods.
	
	Sol:
	#include <stdio.h>
	
	struct Product {
	  char name[30];
	  float price;
	  int quantity;
	};
	
	int main() {
	  // Normal structure variable
	  struct Product p1;
	
	  // Pointer to structure
	  struct Product *ptr;
	
	  // Assign address of p1 to pointer
	  ptr = &p1;
	
	  // Accessing structure members using normal variable (dot operator)
	  printf("Enter product details (Using normal structure variable):\n");
	  printf("Enter name: ");
	  scanf("%s", p1.name);
	  printf("Enter price: ");
	  scanf("%f", &p1.price);
	  printf("Enter quantity: ");
	  scanf("%d", &p1.quantity);
	
	  // Accessing structure members using pointer (arrow operator)
	  printf("\nDisplaying using pointer to structure (ptr->member):\n");
	  printf("Name: %s\n", ptr->name);
	  printf("Price: %.2f\n", ptr->price);
	  printf("Quantity: %d\n", ptr->quantity);
	
	  return 0;
	}
	Scheme: 1.5M for accessing through Normal structure variable and 1.5M through Pointer to structure
		3	5	3

5C	Develop a C program to merge two text files (file1.txt and file2.txt) into a third file (merged.txt). After merging, the program should display the total number of characters and lines in the merged file. Explain how file pointers and end-of-file (EOF) checks contribute to efficient file concatenation. Sol:	4	5	3

```c
#include <stdio.h>

int main() {
    FILE *f1, *f2, *f3;
    char ch;
    int charCount = 0, lineCount = 0;

    // Open files
    f1 = fopen("file1.txt", "r");
    f2 = fopen("file2.txt", "r");
    f3 = fopen("merged.txt", "w");

    if (f1 == NULL || f2 == NULL || f3 == NULL) {
        printf("Error opening one of the files.\n");
        return 1;
    }

    // Copy file1.txt into merged.txt
    while ((ch = fgetc(f1)) != EOF) {
        fputc(ch, f3);
    }

    // Copy file2.txt into merged.txt
    while ((ch = fgetc(f2)) != EOF) {
        fputc(ch, f3);
    }

    // Close files after writing
    fclose(f1);
    fclose(f2);
    fclose(f3);

    // Reopen merged file for counting
    f3 = fopen("merged.txt", "r");

    // Count characters and lines
    while ((ch = fgetc(f3)) != EOF) {
        charCount++;

        if (ch == '\n') {
            lineCount++;
        }
    }

    fclose(f3);
    // Display results
    printf("Merged successfully into merged.txt\n");
    printf("Total Characters: %d\n", charCount);
    printf("Total Lines: %d\n", lineCount);
    return 0;
}
```

Scheme: 2M for Files and merging logic, 1M for Counting Logic, 1 mark for Explanation of EOF checks & File Pointers contribution